Assignment 8

Basic SQL Queries and Aggregate Functions

WHERE, ORDER BY, GROUP BY, HAVING, and COUNT/SUM/AVG/MIN/MAX

Project: LibSphere - Library Management System **Prepared:** 23 October 2026

Platform assumption: MySQL 8.0.16 or later (InnoDB). Outputs in this report are expected results derived from the included sample dataset; run the SQL in MySQL Workbench to capture authentic execution screenshots if required by the instructor.

The following examples are written for the exact sample data in Assignment 7. The output tables show the expected result of running each query after the schema and inserts complete successfully.

Query 1 - Filter records with WHERE

```
-- Query 1: Filter active members using WHERE
SELECT member_id, first_name, last_name, status
FROM Members
WHERE status = 'Active'
ORDER BY first_name;
```

member_id	first_name	last_name	status
1	Asha	Rao	Active
4	Karthik	Iyer	Active
5	Meera	Nair	Active
3	Priya	Sharma	Active
2	Rahul	Menon	Active

Explanation: WHERE filters rows before sorting. The suspended member, Vikram Das, is excluded.

Query 2 - Sort records with ORDER BY

```
-- Query 2: Sort the newest publication years first
SELECT title, publication_year
FROM Books
ORDER BY publication_year DESC
LIMIT 5;
```

title	publication_year
Computer Networks	2012
Sapiens	2011
Clean Code	2008
The Pragmatic Programmer	1999
A Brief History of Time	1988

Explanation: DESC places the most recent publication year first; LIMIT returns only five rows.

Query 3 - Group by category

```
-- Query 3: Count titles in each category with GROUP BY
SELECT c.category_name, COUNT(b.book_id) AS title_count
FROM Categories AS c
LEFT JOIN Books AS b ON b.category_id = c.category_id
GROUP BY c.category_id, c.category_name
ORDER BY title_count DESC, c.category_name;
```

category_name	title_count
Computer Science	3
Fiction	2
Business	1
History	1
Science	1

Explanation: COUNT(b.book_id) counts linked titles per category. LEFT JOIN preserves categories even when none has a book.

Query 4 - Filter groups with HAVING

```
-- Query 4: Keep only categories with at least two titles using HAVING
SELECT c.category_name, COUNT(b.book_id) AS title_count
FROM Categories AS c
JOIN Books AS b ON b.category_id = c.category_id
GROUP BY c.category_id, c.category_name
HAVING COUNT(b.book_id) >= 2
ORDER BY title_count DESC, c.category_name;
```

category_name	title_count
Computer Science	3
Fiction	2

Explanation: HAVING filters aggregated groups after GROUP BY; this differs from WHERE, which filters individual rows.

Query 5 - Aggregate functions

```
-- Query 5: Demonstrate COUNT, SUM, AVG, MIN, and MAX
SELECT COUNT(*) AS title_count,
       SUM(total_copies) AS total_stock,
       AVG(total_copies) AS average_stock_per_title,
       MIN(publication_year) AS oldest_publication_year,
       MAX(publication_year) AS newest_publication_year
FROM Books;
```

title_count	total_stock	average_stock_per_title	oldest_publication_year	newest_publication_year
8	34	4.2500	1813	2012

Explanation: COUNT(*) counts titles; SUM totals all physical copies; AVG calculates average copies per title; MIN/MAX find the earliest and latest publication years in the sample.

Query 6 - Count open loans

```
-- Query 6: Count current loans that are not yet returned
SELECT COUNT(*) AS open_loan_count
FROM Loans
WHERE status IN ('Issued', 'Overdue');
```

open_loan_count
6

Explanation: Issued and Overdue records have no return date and represent books not yet returned in this dataset.

Query 7 - Combine filtering and sorting

```
-- Query 7: Apply a WHERE filter to recent titles, then sort
SELECT title, publication_year
FROM Books
WHERE publication_year >= 2000
ORDER BY publication_year DESC;
```

title	publication_year
Computer Networks	2012
Sapiens	2011
Clean Code	2008

Explanation: Only titles published in or after 2000 are returned, in descending year order.

All result tables are expected outputs from the included seed records, not captured screenshots from a local MySQL Workbench session.